Rather than encode this logic in imperative policy code, we can model it declaratively in OpenFGA using types, relationships, and permissions. Here's how that looks in the schema:

```
type user
  relations
    define manager: [user]

type document
  relations
    define viewer: [user]
    define owner: [user]
    define can_access as viewer or owner or owner->manager
```

Let's break this down:

- The `user` type defines a relationship labeled `manager` between users.
- The `document` type defines two relationships: `viewer` and `owner`. These indicate direct connections between a `document` and `users`.
- The `can_access` permission is defined as a logical expression over those relations:
    - A `user` has permission `can_access` if they are either a `viewer`, an `owner`, or a `manager` of an `owner`.
    - The expression `owner->manager` means for each `user` who is an `owner` of the `document`, follow the `manager` relation on that `user` to identify additional `users` who should have access.

In OpenFGA, relationships are stored as tuples, each representing a single fact. The `#` and `@` symbols delineate the parts of the tuple. A colon separates objects and their type. For example:

```
document:doc1#owner@user:alice
user:alice#manager@user:carol
```

These tuples mean:

- Alice, a `user`, is the owner of `doc1`, which is a `document`
- Carol is Alice's manager